

Intro to deep learning 4

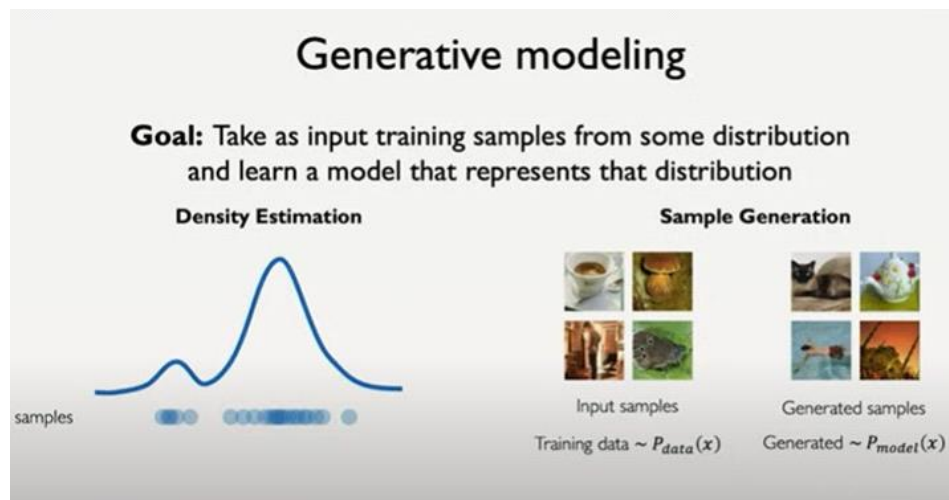
Thursday, August 21, 2025 9:05 AM

Deep Generative Modeling

In this part of the course we turn to **unsupervised learning**

Supervised vs unsupervised learning	
Supervised Learning	Unsupervised Learning
Data: (x, y) x is data, y is label	Data: x x is data, no labels!
Goal: Learn function to map $x \rightarrow y$	Goal: Learn the <i>hidden</i> or <i>underlying structure</i> of the data
Examples: Classification, regression, object detection, semantic segmentation, etc.	Examples: Clustering, feature or dimensionality reduction, etc.

In generative modeling we have training samples and learn a model representing the distribution from which the samples come. Then we can sample from this distribution to generate new data.



How can we **learn a model distribution as close as possible to the data distribution**, i.e. representing well our data?

We do not know the underlying distribution and the training data may be over/underrepresenting some features. Those can be learned in the generative model and identified to debias the training data.

Why generative models? Debiasing

Capable of uncovering **underlying features** in a dataset



Homogeneous skin color; pose

VS



Diverse skin color; pose, illumination

How can we use this information to create fair and representative datasets?



Software Lab!

Outliers can be used in training to improve the generative models. Outliers can be identified from the generative model.

Why generative models? Outlier detection

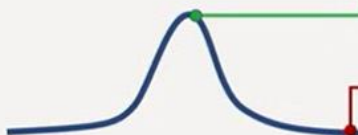
Problem: How can we detect when we encounter something new or rare?

Strategy: Leverage generative models, detect outliers in the distribution
Use outliers during training to improve even more!

95% of Driving Data:
(1) sunny, (2) highway, (3) straight road



Detect outliers to avoid unpredictable behavior when training



Edge Cases



Harsh Weather

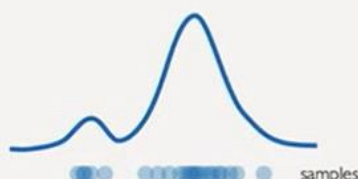


Pedestrians

Generative models can of course create new instances of data.

Why generative models? Sample generation

- **Generative models learn probability distributions**
- **Sampling** from that distribution → new data instances
- **Backbone of Generative AI:** generate language, images, and more



Natural Language

★ 6.S191 Guest Lectures!



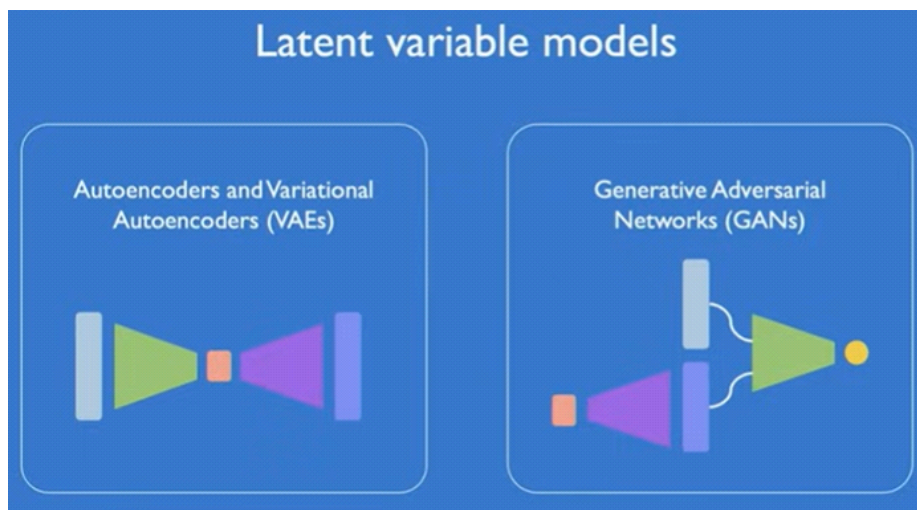
Images & Videos



Biology

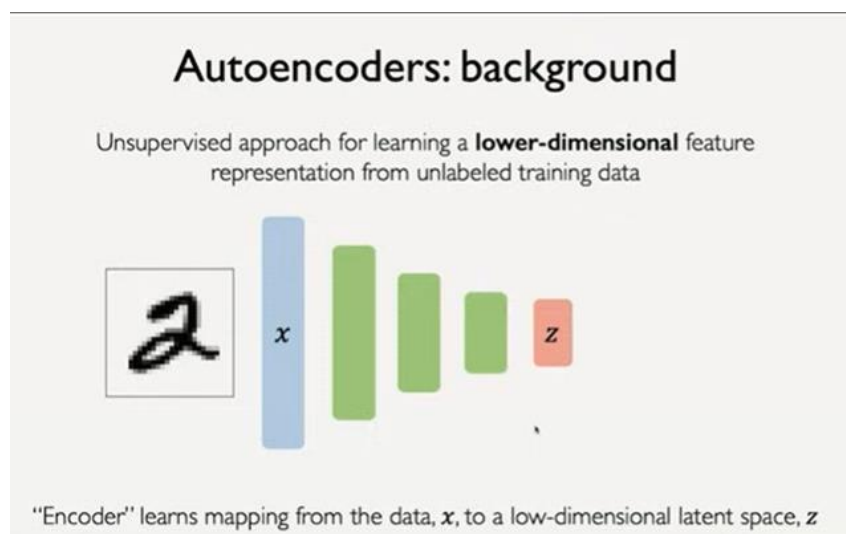
★ 6.S191 Guest Lecture!

We focus on two foundational models called latent variable models



Autoencoder

They learn a low-dim feature space (latent space) which represent the data through subsequent layers. The latent vector compresses the data in the essential features.

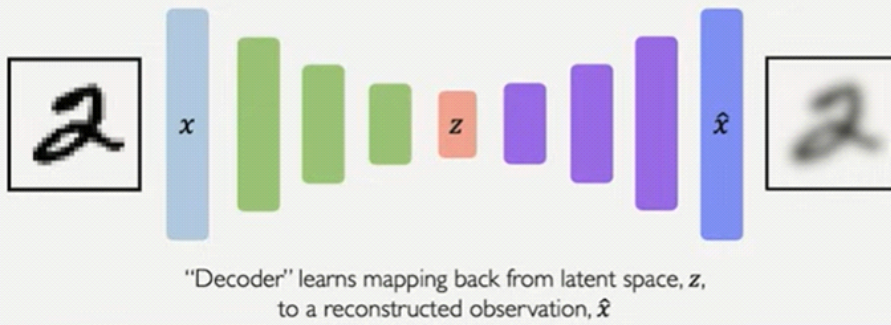


By not assuming any label, we try to reconstruct the input with decoding. We can compare input and output and train the network to minimize differences between images. The loss function **does not require thus any label**.

Autoencoders: background

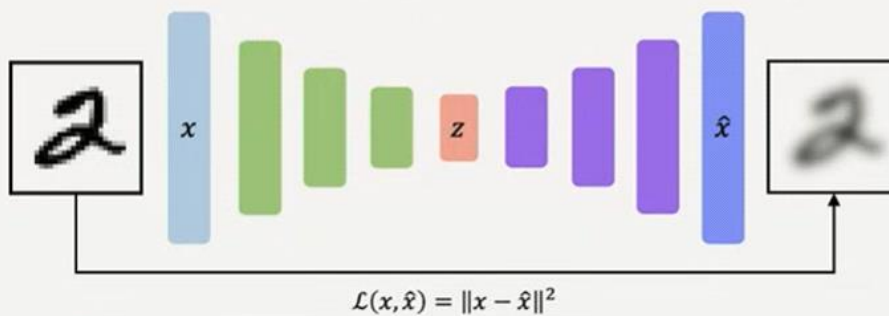
How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



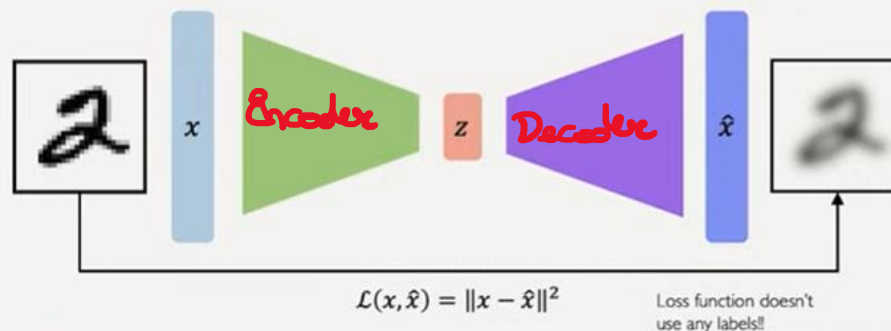
How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**

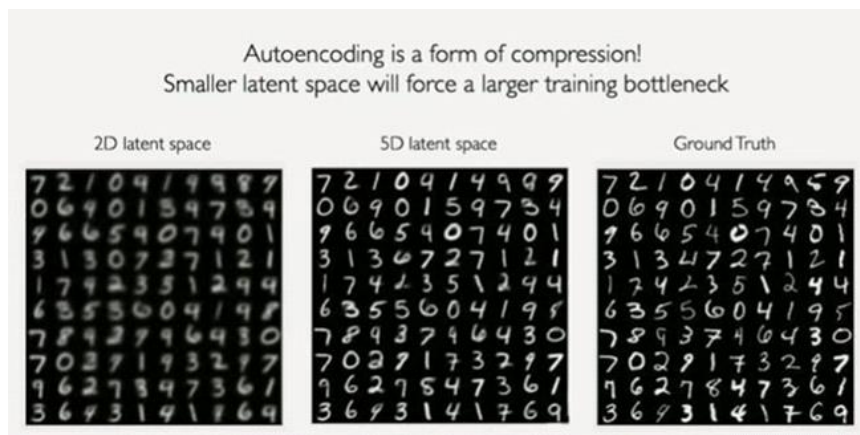


How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



The dimensionality of latent space matters. Too much compression forces a worse reconstruction.



This hidden layer bottleneck compresses the data, and we use reconstruction loss to best capture data features. This is what an autoencoder does.

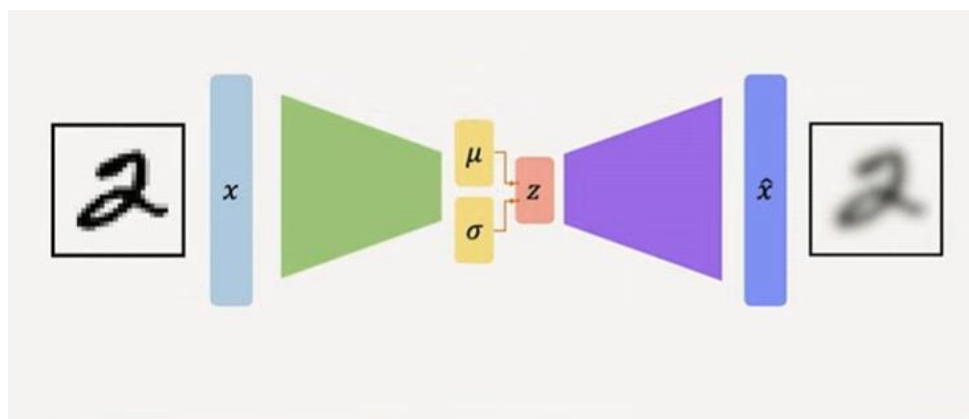
Bottleneck hidden layer forces network to learn a compressed latent representation

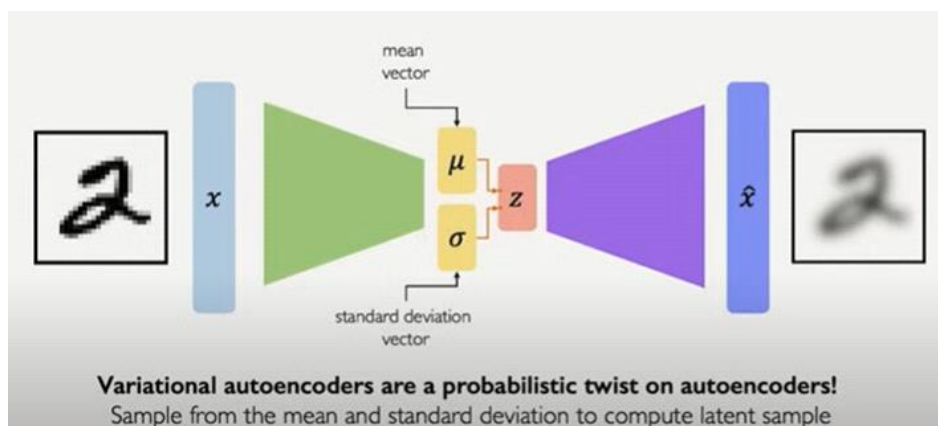
Reconstruction loss forces the latent representation to capture (or encode) as much "information" about the data as possible

Autoencoding = **Automatically encoding** data; "Auto" = **self**-encoding

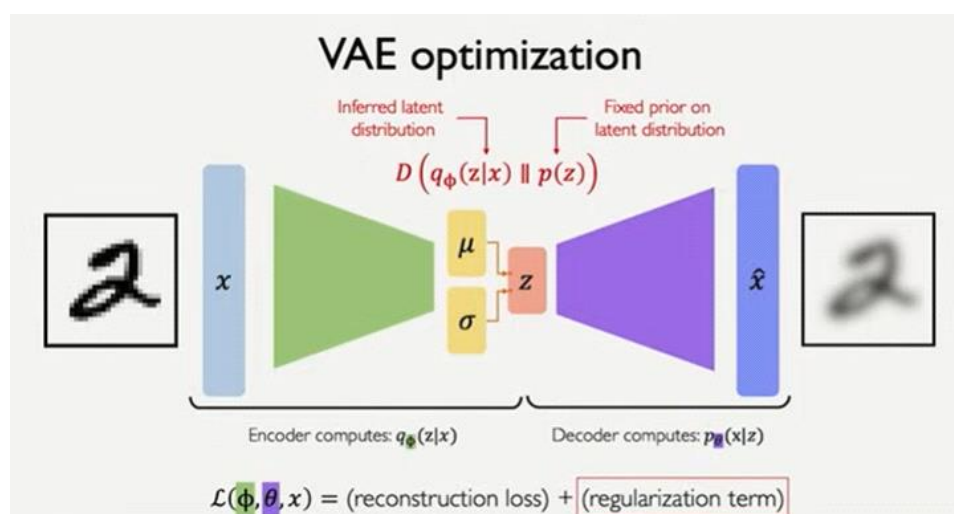
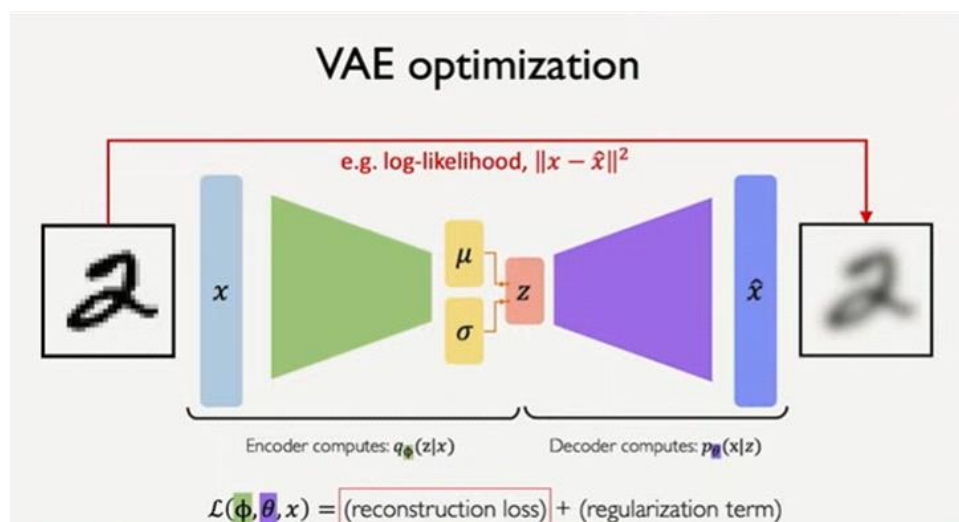
Variational Autoencoder (VAE)

Usually a **Variational** autoencoder is applied to introduce diversity for generating the data from input. With the introduction of some randomness the layer z is no longer deterministic. We parametrize instead each latent variable with a distribution with *mean* and *standard deviation*. In this way we sample from those parameters to generate data instances.





The encoder computes the prob distribution of the latent space given the data and the decoder does exactly the opposite switching data and latent space. The loss changes as well, with reconstruction loss + regularization term. The regularization involves a prior for z and a distance between estimated distribution of $z|x$ and the prior.



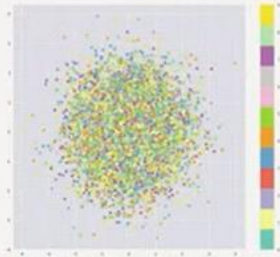
How does the prior work

Often people choose a standard normal because the latent parameters will avoid to cluster points together in the latent space doing overfitting. The regularization distance is the KL-divergence.

Priors on the latent distribution

$$D(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_{j=0}^{k-1} (\sigma_j + \mu_j^2 - 1 - \log \sigma_j)$$

KL-divergence
between the two
distributions



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

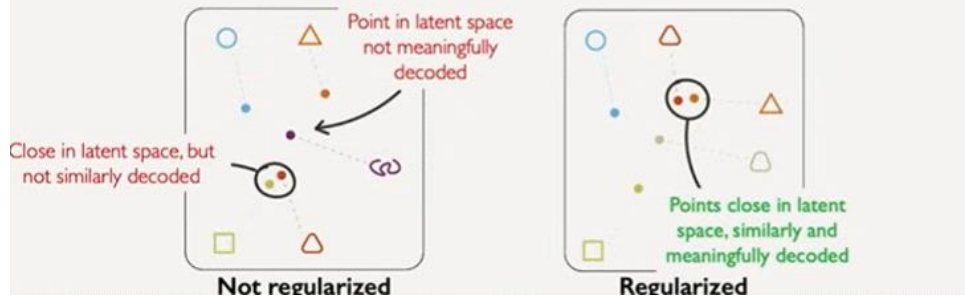
- Encourages encodings to distribute evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (i.e., by memorizing the data)

But why regularization and normal prior? Remember that closeness in latent space means closeness in input similarity (**continuity**). Also we want evenly parametrized latent space to being able to sample meaningfully (**completeness**). Not using regularization might break those two properties.

Intuition on regularization and the Normal prior

What properties do we want to achieve from regularization? 🤔

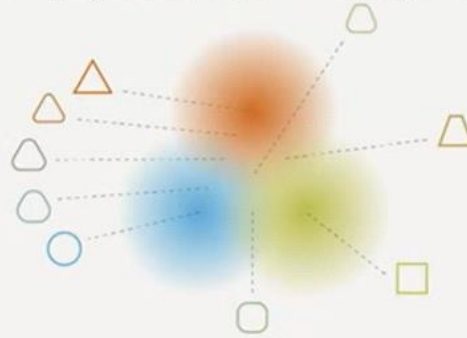
1. **Continuity:** points that are close in latent space → similar content after decoding
2. **Completeness:** sampling from latent space → “meaningful” content after decoding



Remember, using distribution does not guarantee the two properties. But using normal distribution as a prior makes it computationally doable and ensure variability is regularized.

Intuition on regularization and the Normal prior

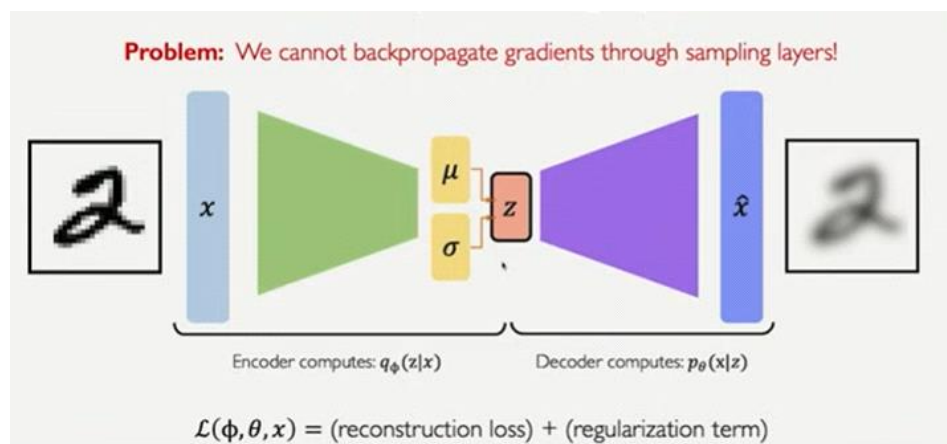
1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ "meaningful" content after decoding



Regularization with Normal prior helps enforce **continuity & completeness** in the latent space.

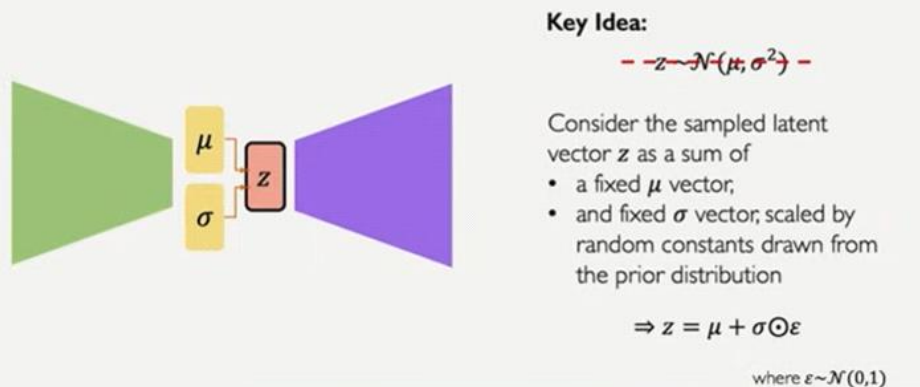
VAE training

Backpropagation requires a deterministic NN layer which we do not have

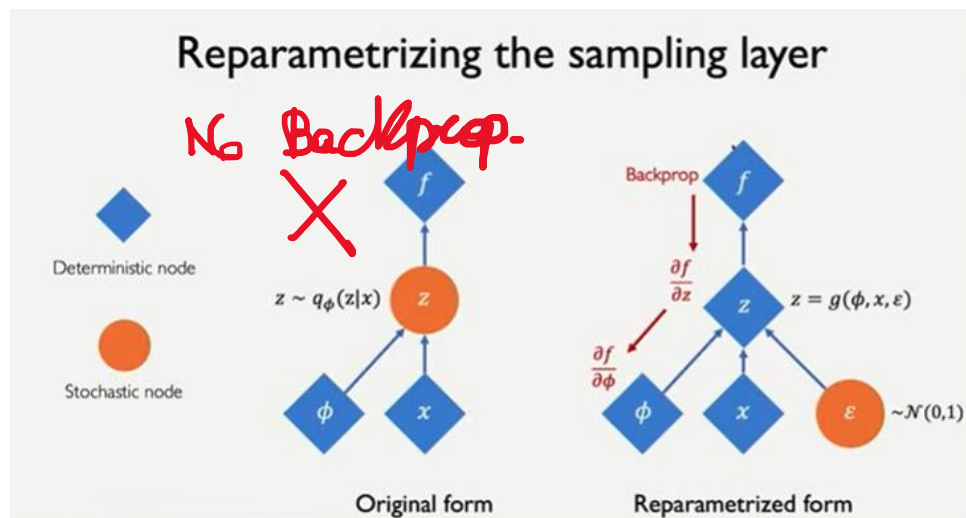


We do a change of the sampling operation to use backpropagation. We parametrize the sampling step. The latent z is the sum of deterministic mean and sigma vectors, where sigma is scaled by a random gaussian variable.

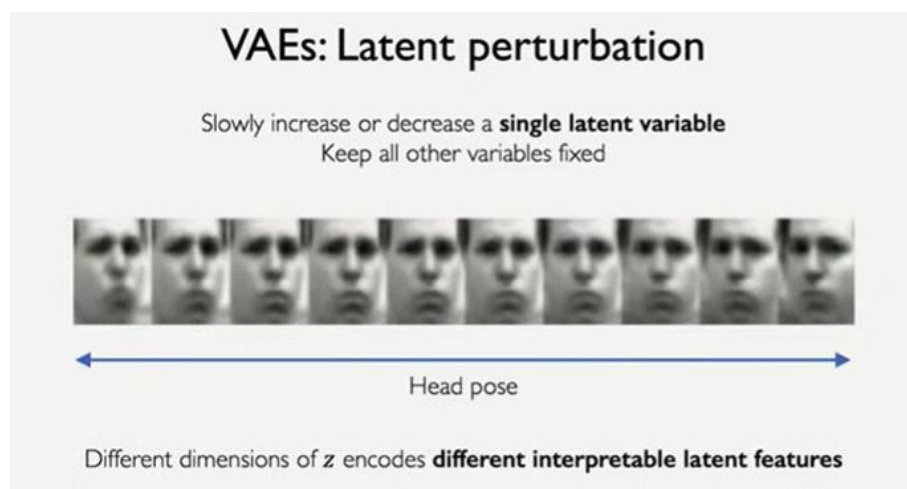
Reparametrizing the sampling layer



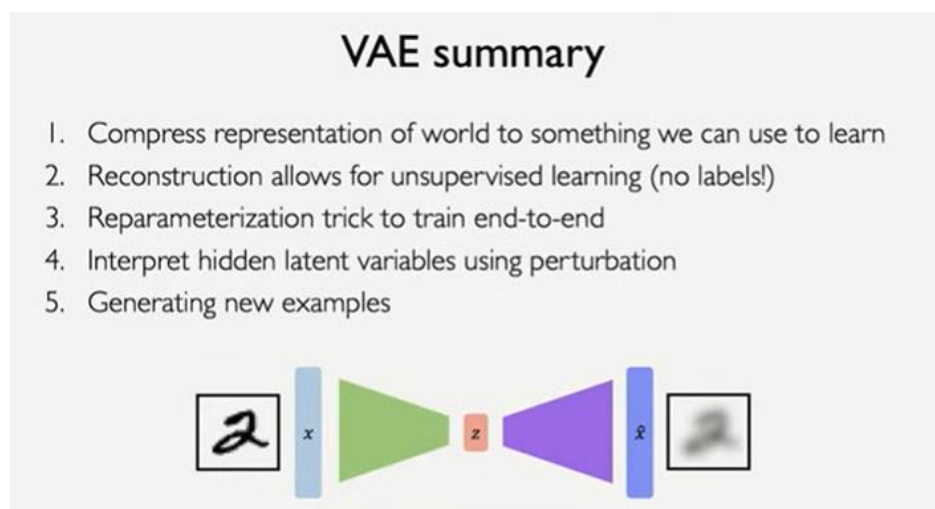
How does this affect backpropagation?



The latent variable space is changed by a specific change in input which can be identified.



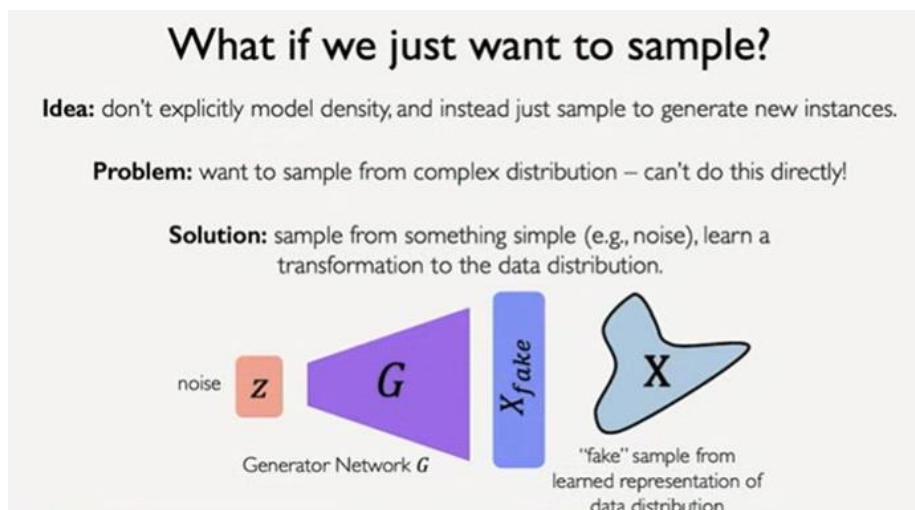
Wrapup



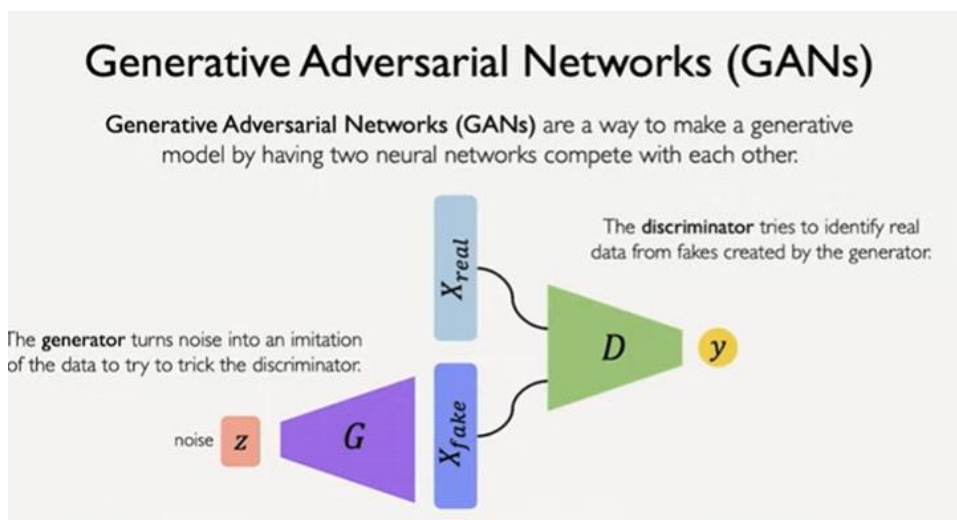
Generative adversarial network (GAN)

This model care only about generating data similar to the input. We do not model the

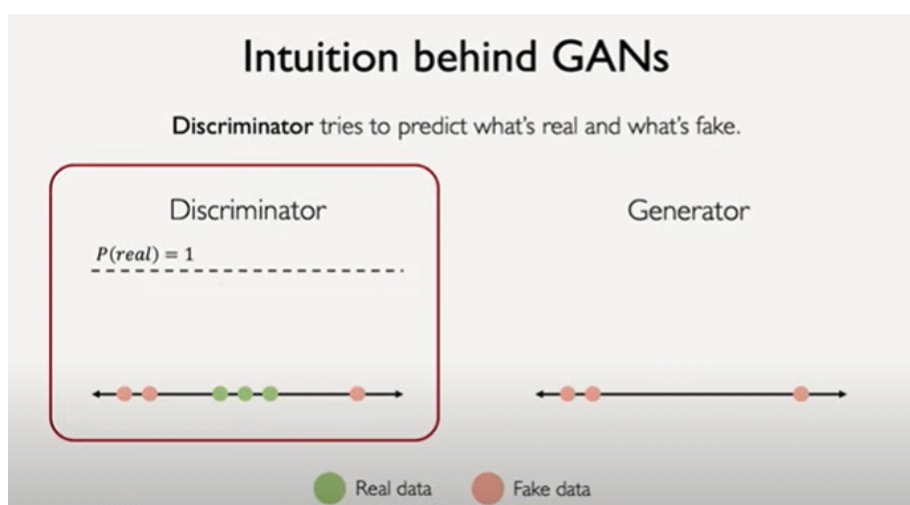
density directly. It learns a distribution for the data.

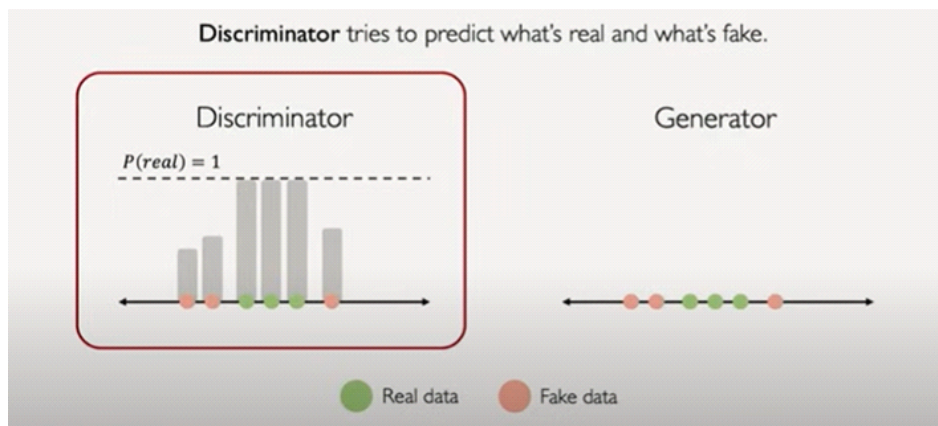


A GAN uses specifically two networks competing. One creates outputs and the other discriminates generated outputs to see if they are like the input or not.

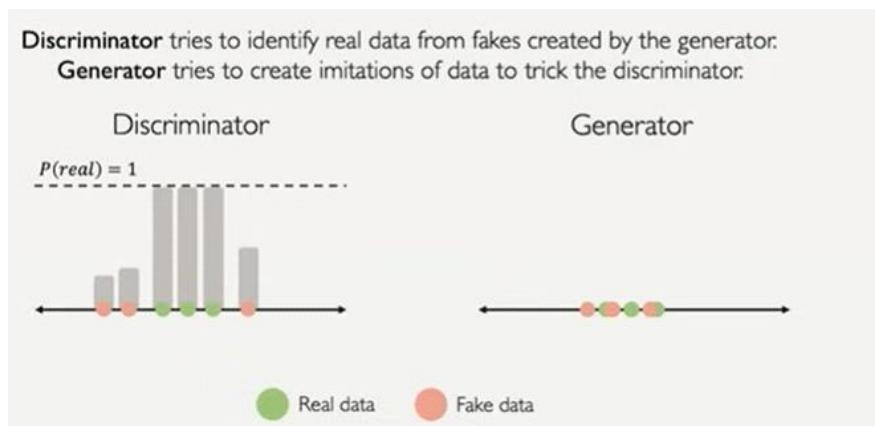


Example: the generator is trained on noise and generates fake data. Then the discriminator is trained until it distinguishes best fake and real data.

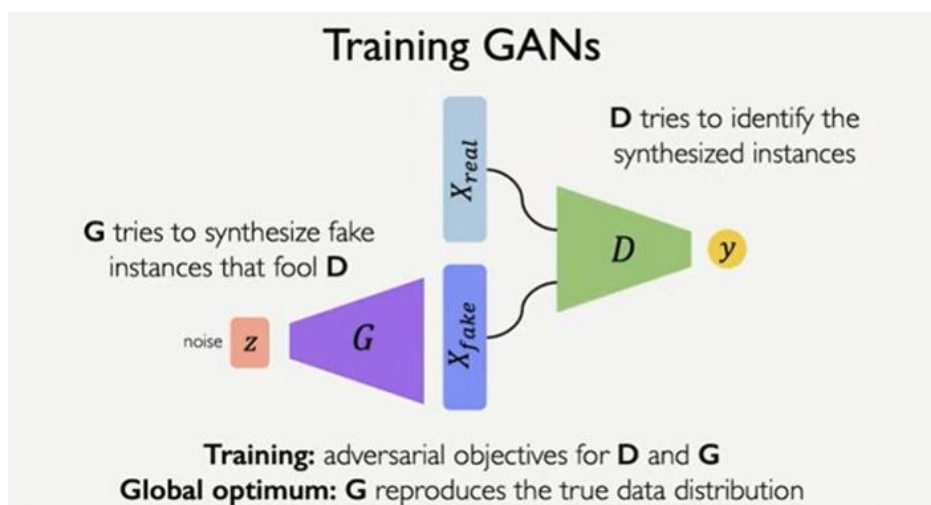




Again the generator over the steps is trained to make the best fake data which the discriminator is no longer really good at distinguishing from each other

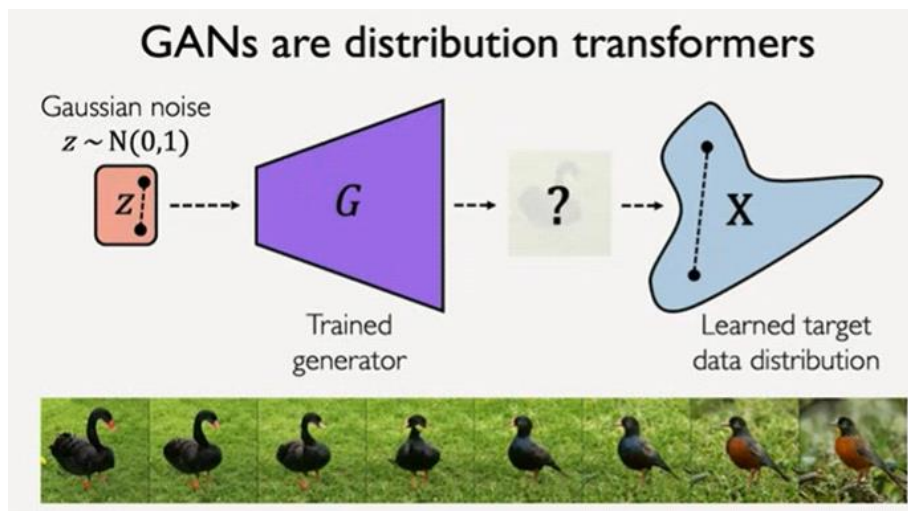


After training we are able to generate data very similar to the input.

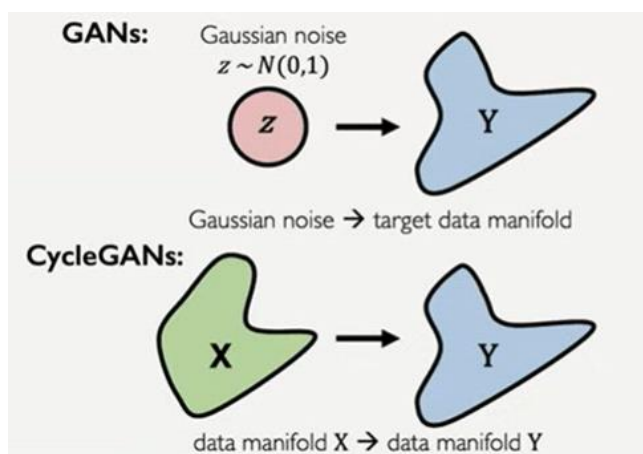
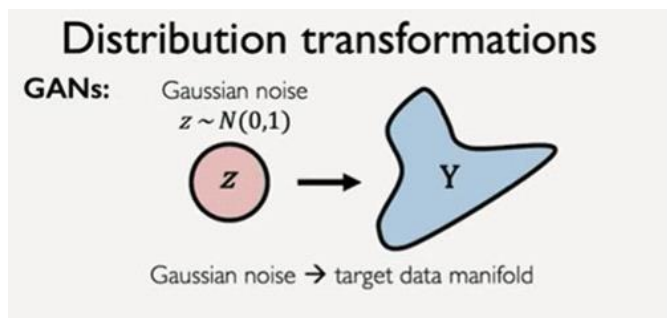


You can think about GANs as a tool to learn a distribution starting from noise, learning a mapping from noise to the target distribution. Giving smooth varying points of the learned distribution gives smooth interpolation of varying images!

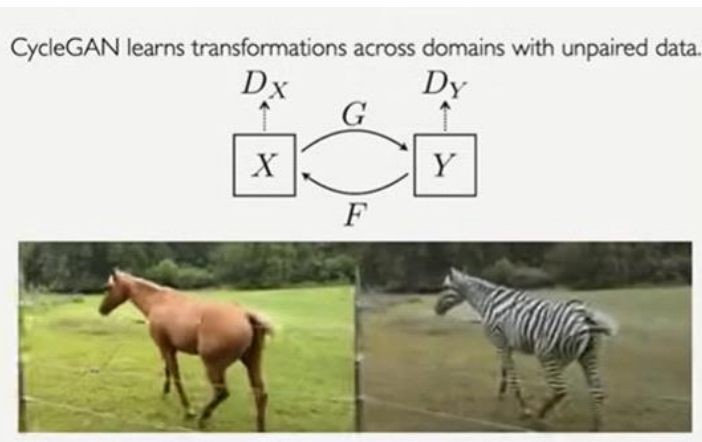




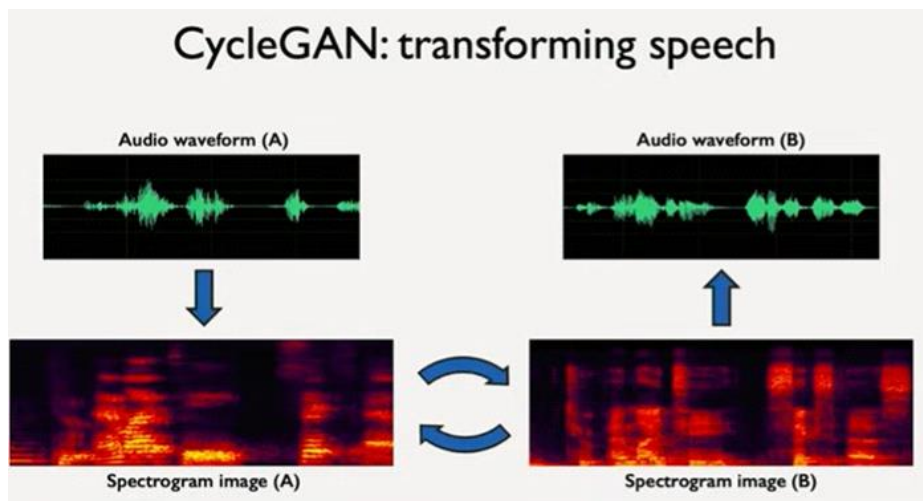
You can start also from a specific data manifold to move into another



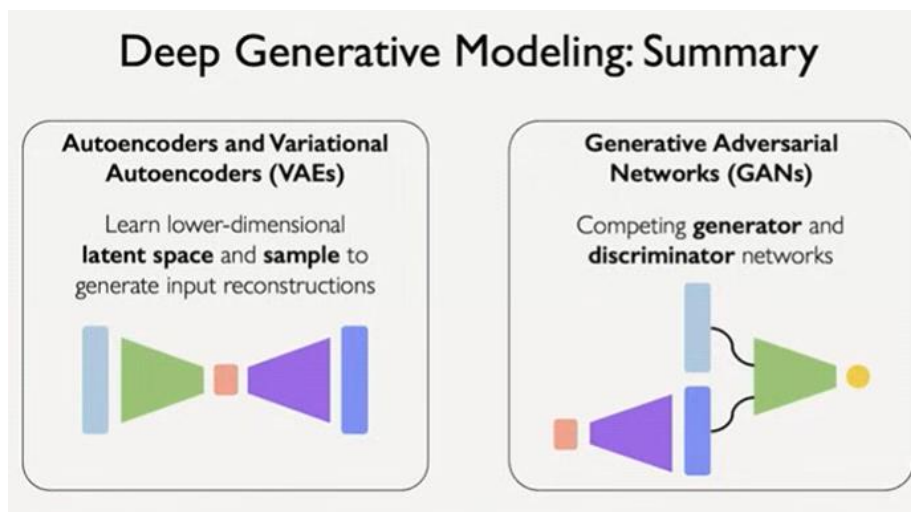
This leads to changes from one style to another in images for example. Cyclic GANs have two sets of generator/discriminators.



You can also transform speech.



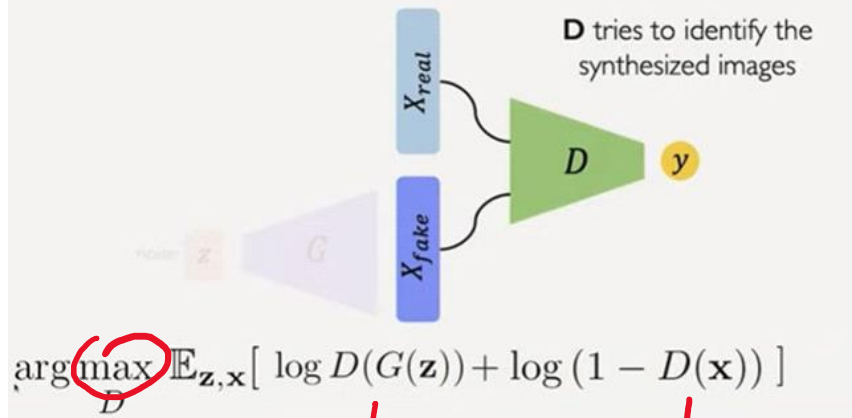
Wrapup



Some extra about loss

We want to optimize jointly generators and discriminators. Looking at discriminator

Training GANs: loss function

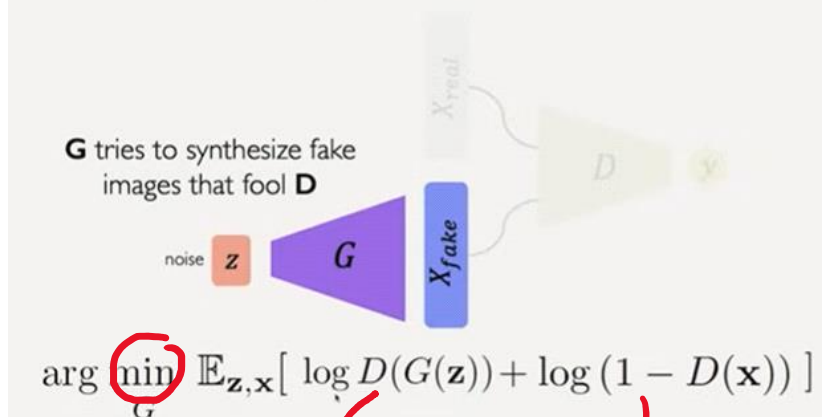


Maximizing: Best detect the fake images

Best detect the true input

and at generator

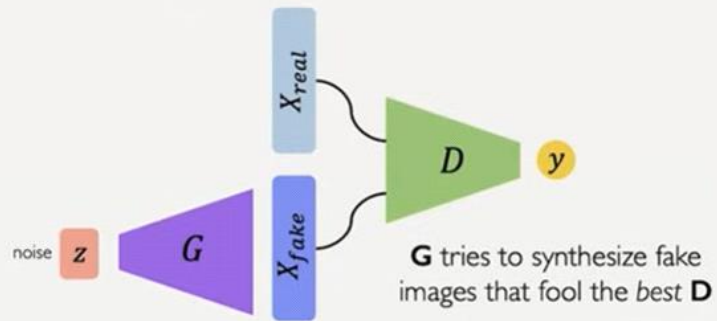
Training GANs: loss function



minimizing that data is identified as real or fake

All together

Training GANs: loss function



$$\arg \min_G \max_D \mathbb{E}_{\mathbf{z}, \mathbf{x}} [\log D(G(\mathbf{z})) + \log (1 - D(\mathbf{x}))]$$

Loss is differentiable but it can be very hard to do it in practice.